

Projektets olika ansvarsområden - Steg för steg instruktioner

Börja med att

1. Utgå från starter-app-template och kör npm install.
2. Skapa en .env. Alla i gruppen behöver en egen .env fil med följande variabler:
 - a. MONGODB_URL,
 - b. JWT_SECRET,
 - c. NODE_ENV
 - d. CLIENT_URL.
3. .gitignore följande mappar/filer:
 - a. node_modules
 - b. .env
 - c. .DS_Store
4. Avkommentera anslutningen till MongoDB i api/index.ts och kontrollera att servern startar.
5. Lägg upp mapparna src/models, src/controllers och src/routes, med en fil per tabell i varje. Registrera era routers i api/index.ts.

Ansvarsområde 1 - users, registrering och inloggning

API

1. Skapa modellen för users med fälten som står i uppgiften.
2. Bygg endpointen som registrerar en användare. Kontrollera att både användarnamn och lösenord kommit med i anropet, hasha lösenordet med bcrypt, och spara användaren med hashen.
 - a. Innan ni registrerar och sparar ny användare i DB, kolla om användarnamnet redan är taget. Om det redan är taget, returnera statuskod 409 tillsammans med ett passande felmeddelande ex "Username is already taken"
 - b. Om användaren registrerades, se till att svaret inte innehåller lösenord hashen. Skickar ni tillbaka hela användarobjektet följer den med. Bygg svaret för hand med bara de fält ni vill visa.
3. Bygg endpointen som loggar in en användare. Hämta användaren efter username, jämför lösenordet mot hashen med bcrypt, signera en token och lägg den i en httpOnly-cookie precis som i föreläsning 20.
4. Skapa hanteringen för endpointen POST /auth/logout, som rensar cookien.
5. Testa 2–4 i Insomnia, innan ni rör klienten. Det är mycket lättare att felsöka där.
6. Bygg endpointen för att hämta alla användare och en enskild användare, båda skyddade via verifyToken. Plocka bort lösenordsfältet ur svaret:

```
const users = await User.find().select('-password')
```

7. Bygg uppdatera och radera, båda skyddade via verifyToken. Vid uppdatering ska bara de fält som faktiskt skickats med ändras, så att en PATCH inte nollställer resten. Byter någon lösenord måste det hashas om innan det sparas.

Klient

8. Bygg klientens registrera-sida & login-sida med varsitt formulär. Ta med **credentials: "include"** i varje anrop, annars sparas och skickas aldrig cookien.
9. Bygg den lösenordsskyddade sidan. Ni kan inte läsa cookien från JavaScript, det är hela poängen med httpOnly. Istället frågar ni servern genom att be om något skyddat och titta på statuskoden. Får ni 401 eller 403 skickar ni användaren till inloggningssidan. Det är exakt samma mönster som protected.js i föreläsning 20. Om användaren är inloggad, visa en lista av alla användare i adminpanelen.

Ansvarsområde 2 - books

API

1. Skapa modellen för books med fälten som står i uppgiften. Stäm av med ansvarsområde 3, boken behöver kunna hämtas tillsammans med sina reviews.
2. Bygg endpointen som hämtar alla böcker. Den ska vara öppen, eftersom besökare ska kunna se boklistan utan att logga in.
3. Bygg endpointen som hämtar en enskild bok med tillhörande reviews. Det är samma koppling som todos och subtasks i föreläsning 18. Gör den här biten tillsammans med ansvarsområde 3.
4. Bygg skapa, uppdatera och radera, alla tre bakom verifyToken.
5. När ni raderar en bok, radera även bokens reviews. MongoDB gör det inte automatiskt. Annars blir det kvar reviews som pekar på en bok som inte finns.
6. Om ni väljer att göra en separat sida för bokhanteringen i adminpanelen behöver ni en skyddad variant att fråga. Den öppna boklistan kan inte skydda något, eftersom besökare måste komma åt den. Lägg till en skyddad route — och observera ordningen:

```
router.get('/', fetchAllBooks)           // öppen
router.get('/admin', verifyToken, fetchAllBooks) // skyddad
router.get('/:id', fetchBook)
```

/admin måste ligga före /:id, annars matchar Express /:id med id = "admin" och ni får ett förvirrande 404.

Har ni istället en enda adminpanel med både användare och böcker behöver ni inget extra, anropet till användarlistan skyddar hela sidan.

Klient

7. Bygg besökarnas boklista i klienten, där varje bok länkar vidare till sin egen sida med bokens id i adressfältet.
8. Bygg listan över böcker i adminpanelen.
9. Bygg formuläret för att skapa en ny bok. Tänk på att genres är en array i databasen men ett textfält i formuläret, bestäm hur ni tar emot flera genrer och dela upp texten innan den sparas.

Ansvarsområde 3 — reviews

API

1. Skapa modellen för reviews. Kopplingen till boken fungerar precis som `todo_id` på subtasks i föreläsning 18 - barnet håller förälderns id.
2. Koppla ihop bok och reviews så att en bok kan hämtas med alla sina reviews i ett enda anrop. Gör det tillsammans med ansvarsområde 2, eftersom det kräver en ändring i bokmodellen.
3. Bygg endpointsen för att hämta alla reviews och en enskild review. Båda öppna.
4. Bygg endpointen för att skapa en review. Den ska vara öppen, vem som helst ska kunna skriva en recension utan konto. Kontrollera att betyget ligger mellan 1 och 5, och att boken faktiskt finns innan ni sparar.
5. Bygg uppdatera och radera, båda bakom `verifyToken`.

Klient

6. Bygg boksidan i klienten. Den läser bokens id ur adressfältet, samma mönster som `todo.html` i föreläsning 18. Sidan har tre delar: bokens uppgifter med en tillbaka-länk, formuläret för en ny review, och listan med bokens reviews.
7. När en ny review sparats, hämta boken igen med tillhörande reviews, så att review listan visar den nya reviewen.

Att tänka på

- Ansvarsområde 2 och 3 måste komma överens om kopplingen mellan bok och review. Gör den biten tillsammans.
- Under tiden ni väntar på varandra kan ni bygga klart era API delar, samt bygga klienten med hårdkodad testdata i HTMLen.